From VLM to VLA

Student edition — from VLM to VLA, the essentials

Professor Anis Koubaa • 14–16 July 2026

Why is making a robot *act* so hard?

Hand-code every case

- The tomato is never in the same spot twice.
- Lighting, leaves, ripeness all change.
- You'd need an `if` for *every* situation — impossible.

Learn from examples

- Show the robot a few dozen good picks.
- It learns the *pattern*, not a fixed script.
- New tomato position? It still works.

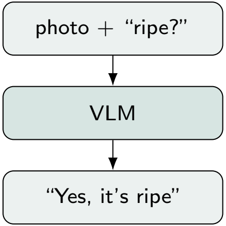
Describing needs **one answer**; *acting* needs the **right move** hundreds of times, in a world that never looks the same. **So robots learn from demonstrations.**

A 30-second recap: what is a VLM?

A VLM (Vision–Language Model) takes an **image + a question** and answers in **words**.

- *You:* (photo of a plant) "Is this tomato ripe?"
- *VLM:* "Yes — it's fully red and ready to pick."

It **understands** pixels and language together — but it can only **talk**, not **move**.



A **VLA** keeps this understanding — and adds the *moving*.

Learning Objectives

By the end of this lecture you will be able to:

- Say what a **VLA** is and how it differs from a VLM.
- Name the **four things a robot records**: what it *sees*, its *state*, its *action*, its *instruction*.
- Read and understand a **robot dataset** (LeRobot format).
- Explain how a VLA **learns** from that data — imitation, not RL.
- Read a **sample training script** and know what it optimizes.

Ingredient 3 of 4 — what is an “action”?

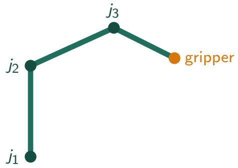
An **action** is just the *numbers that move the robot*.
For the `UR5e` arm:

- `6 joint targets` — shoulder, elbow, wrists . . .
- `1 gripper` value — open or close.

So one action is a vector:

$$a = [\, j_1,\, j_2,\, j_3,\, j_4,\, j_5,\, j_6,\ \text{grip}\,]$$

A **motion** = many such vectors over time — we log $\sim$**30 per second**.



The model’s job: output the next a.

A VLA doesn’t “decide to pick” in words — it outputs **these numbers**, step after step.

Put the pieces together: a robot *dataset*

You now know all four ingredients. At every tick we bundle them into one **frame**:

field	**example**	**meaning**
`observation.images.wrist`	(480,640,3) image	what the camera *sees*
`observation.state`	6 joint angles	where the arm *is*
`action`	6 next joint targets	what you *did* (the label!)
`task`	"pick the ripe red tomato"	what to do, in *words*

A sequence of frames = one **episode** (one pick). Many episodes = a **dataset**.

- **Good data beats a bigger model**: record $\sim$50 episodes, $\sim$10 per variation (e.g. tomato position), images 224×224, action & state aligned, no NaNs.

How do we collect it? (teleoperation)

- A **demonstration** = a human doing the task once, *recorded*.
- **Teleoperation** = you drive the robot (keyboard / joystick / leader-arm) while we log *every frame* (the four ingredients, $30\times/\text{s}$).
- One recorded run = one **episode**. Repeat $\sim$50 times $\Rightarrow$ your dataset.

You are **not writing motion code** — you are *showing* the robot, like guiding a child's hand. The robot will later **copy what you showed**.

How can one model output a *movement*?

A language model predicts the **next word** as a number from its vocabulary. A `VLA` predicts the **next action** the very same way:

image + "pick the tomato" → VLA → action numbers (tokens or floats) → motors

- `OpenVLA`: chop each joint into **256 bins** → predict "action words" (discrete).
- `SmolVLA`, π_0: a small **action expert** outputs the numbers directly (continuous).

Predicting an action is just like predicting a word.

Imitation learning, step by step

1. **Collect** demonstrations by teleop → many *(image, state, action)* frames.
2. The **action you did** is the "right answer" — the **label**.
3. **Train** the model to *predict that action* from the image + state + instruction, shrinking the gap between its guess and your action.
   (This "copy the expert" training has a name: **behavioral cloning**.)
4. **Deploy** (closed loop): predict an action → execute → look again → repeat.

1. collect demos → 2. action = label → 3. train to predict it → 4. deploy closed loop

The whole loop: teach by showing, command by speaking

Morning
teleop the UR5e,
record 30 tomato picks

→

train (imitation)
SmolVLA copies
the demos

→

Afternoon
type "pick the ripe
red tomato" $\rightarrow$ it moves

- No motion is hand-coded — the skill is **learned from your demonstrations**.
- The same demos + a new instruction $\Rightarrow$ the robot generalizes to nearby situations.
- **This is the whole shift**: from *programming* robots to *teaching* them.

Key Takeaways

- **VLA** = image + instruction → actions (a VLM that learned to move).
- A robot records **four things**: what it *sees*, its *state*, its *action*, its *instruction* — bundled per frame, aligned in time.
- Those frames form a **dataset**; **quality** > **quantity** > **model size**.
- A VLA **learns by imitation** (behavioral cloning) — not RL.
- Recording and training are **a few lines** — the lab is mostly *good demos*.

Questions?

Next: *Lab 2.1 — teleoperate the UR5e & record your own dataset.*